

SQLens

An Educational SQL Compiler and Query Analysis Visualizer

Compiler Design Laboratory (BCSE307P) — Review 2: Implementation Progress Review

STUDENT DETAILS

- **Candidate Name:** Mishti Parikshit Gandhi
- **Registration Number:** 24BCE0881
- **Course & Code:** Compiler Design Lab (BCSE307P)
- **Laboratory Slot:** L49 + L50

ACADEMIC & REVIEW DETAILS

- **Faculty Reviewer:** [Faculty Name / Reviewer]
- **Department:** School of Computer Science and Engineering (SCOPE)
- **Institution:** Vellore Institute of Technology, Vellore
- **Review Stage:** Phase 2 / Review 2 Evaluation (October 2026)

Project Overview: Concept, Purpose & Current Status

What is SQLens & What Problem Does It Address?

System Concept & Core Motivation

- **Domain-Specific Compiler:** SQLens is an educational compiler built in C++ using Flex and Bison for a declarative MiniSQL language.
- **Bridging Two Core Disciplines:** Connects Compiler Design principles (lexing, LALR parsing, ASTs, type checking) with Database Systems (relational algebra, optimization trees, physical scans).
- **Full Pipeline Transparency:** Replaces traditional black-box query compilers by exposing every transformation stage from source query text to final tabular records.
- **Zero Mock Outputs:** All representations, token streams, AST nodes, relational plans, and row traces are produced by the real compiled C++ executable (minisql.exe).

Review 2 Implementation Status

Verified Engineering Achievements

- **Phase 1 Frontend (100% Complete):** Lexical scanner (Flex), LALR(1) parser (Bison), AST hierarchy, symbol table catalog, and semantic type checker.
- **Phase 2 Backend (100% Complete):** Canonical Relational IR generator, algebraic query optimizer, and in-memory CSV execution engine.
- **Automated Test Suite (100% Pass Rate):** 26 automated unit and integration tests (26/26 passing) covering projections, filters, logic, ordering, limits, and fail-stop errors.
- **Interactive Web Workbench (~80% Complete):** React 18 / TypeScript frontend and Node.js API bridge providing 10-stage animated execution and viva theory guides.

Revised Problem Statement & Educational Need

The Problem in Conventional Compiler & Database Instruction

Pedagogical Challenges in Computer Science Curricula

- **Imperative Language Focus:** Academic compiler courses typically focus on C or Java subsets, making parsing and code generation feel abstract and disconnected from data-intensive systems.
- **Database Black Boxes:** In DBMS courses, students learn relational algebra expressions, query execution plans, and cost estimation theoretically, but never see how raw declarative SQL queries are tokenized, syntax-checked, and converted into relational algebra trees.
- **Inaccessible Monolithic Toolchains:** Industrial toolchains like GCC or LLVM are too complex for laboratory inspection, preventing students from stepping through AST construction, semantic verification, and optimization rewrites.

Formal Project Problem Statement

Official Formulation for Review 2

“Compiler Design concepts can be difficult for students to understand because the internal transformations performed during compilation are not always visible. SQLens addresses this problem by implementing a lightweight MiniSQL compiler and exposing its query-processing pipeline through an interactive visual interface. It enables learners to inspect tokens, syntax structures, semantic analysis, intermediate representation, optimization, execution, and errors through one integrated workflow.”

- **Core Educational Value:** Turns abstract theoretical compiler phases into concrete, observable artifacts.
- **Integrated Error Isolation:** Exposes exactly which compilation phase rejects invalid code, reinforcing compiler theory.

Project Objectives & Scope of Implementation

1. Lexical & Syntax Parsing

Implement a formal MiniSQL scanner with Flex and an LALR(1) context-free grammar with Bison to parse declarative queries and establish operator precedence.

2. AST & Symbol Catalog

Construct a strongly-typed Abstract Syntax Tree and maintain an in-memory catalog schema verifying relation names, attribute existence, and column types.

3. Semantic Validation

Implement strict semantic analysis enforcing table resolution, column scoping, operator type-compatibility, and boolean predicate validation.

4. Relational Intermediate Code

Translate validated ASTs into canonical Relational Algebra Intermediate Representation (IR) trees comprising Scan, Filter, Project, Sort, and Limit operators.

5. Rule-Based Query Optimization

Implement algebraic query transformations including Filter Pushdown, Late Projection (projection reordering), and Limit Early-Stopping.

6. CSV Execution & Fail-Stop Errors

Execute physical relational plans against CSV datasets and enforce fail-stop error isolation to halt execution at the exact stage of failure.

Scope & Limitations: Supported Syntax vs Out-of-Scope

Supported MiniSQL Language (Verified in Code)

Current Grammar & Engine Capabilities

- **Query Structure:** Declarative SELECT ... FROM ... [WHERE ...] [ORDER BY ...] [LIMIT ...];
- **Target Relation:** Single relation catalog over 'students' (id: INT, name: STRING, cgpa: FLOAT, age: INT).
- **Attribute Projection:** Single and multi-column projections (e.g., SELECT name, cgpa).
- **Comparison Predicates:** Binary comparisons (=, !=, <, >, <=, >=) over numeric, string, and boolean attributes.
- **Logical Conjunctions:** Compound boolean expressions combining AND, OR, and parenthesized sub-expressions.
- **Sorting Operations:** ORDER BY <column> [ASC | DESC], supporting sorting on unprojected attributes via Late Projection.
- **Row Cardinality:** LIMIT <integer> with zero and bound checks.
- **Data Source:** CSV streaming storage engine parsing typed comma-separated records.

Explicitly Out-of-Scope / Phase 3 Planned

Honest Boundaries of MiniSQL Prototype

- **Multi-Table Operations:** Relational joins (INNER JOIN, CROSS JOIN) are intentionally excluded from Phase 2; planned for Phase 3.
- **Aggregate Functions:** Functions such as COUNT(), SUM(), AVG(), MIN(), MAX(), and GROUP BY / HAVING clauses are not implemented.
- **DML & DDL Statements:** Modifying statements (INSERT, UPDATE, DELETE) and schema creation (CREATE TABLE) are excluded; focus is purely on query compilation.
- **Nested Queries:** Correlated and uncorrelated subqueries are out of scope.
- **Multi-File Catalogs:** Dynamic schemas across multiple arbitrary CSV files are not supported in the Phase 2 baseline.

Overall System Architecture & Data Flow

1. User & Interface Layer

Presentation & API Bridge

- **Web Workbench:** React 18 + TypeScript + Vite UI running locally on port 5173.
- **Interactive Controls:** SQL editor with 9 presets, 10-stage animated pipeline stepper, and AST/IR visualizers.
- **API Bridge Server:** Node.js Express server on port 3001 routing /api/compile.
- **ChildProcess Invocation:** Backend executes minisql.exe --json '<query>' and captures pure JSON stdout.

2. Core C++ Compiler Pipeline

10-Stage Compilation Engine

- **[01] Source Query:** Raw SQL text validation.
- **[02] Lexical Analysis:** Flex scanner (lexer.l) emitting token stream.
- **[03] Syntax Analysis:** Bison parser (parser.y) enforcing LALR(1) grammar.
- **[04] AST Construction:** Typed C++ AST hierarchy (ast.cpp).
- **[05] Symbol Table Lookup:** Schema metadata verification (symbol_table.cpp).
- **[06] Semantic Analysis:** Strict type and scoping validation.
- **[07] Relational IR:** Canonical algebra operator tree.
- **[08] Rule Optimizer:** Filter pushdown & late projection rewrites.
- **[09] CSV Execution:** Physical tuple evaluation (executor.cpp).
- **[10] Query Result:** Formatted rows with high-res stage timings.

3. Storage & Error Flow

Data Source & Diagnostics

- **Storage Source:** data/students.csv containing typed student records.
- **In-Memory Evaluation:** Tuples parsed, filtered, sorted, and projected in RAM.
- **Strict Fail-Stop Trapping:** If an error occurs at any stage, pipeline halts immediately.
- **Structured JSON Output:** Stage timings and diagnostics emitted for UI rendering.

Frontend Compilation: Lexical & Syntax Analysis

Lexical Analysis: Flex Scanner (lexer.l)

Token Identification & Regular Expressions

- **DFA Token Recognition:** Uses Flex regular expressions to recognize SQL keywords (SELECT, FROM, WHERE, ORDER, BY, LIMIT), symbols, and literals.
- **TokenRecord Structure:** Captures token symbol, raw lexeme, token classification, and line number for every extracted token.
- **Lexical Error Trapping:** Catches unrecognized characters (e.g. '@', '\$') via fallback rules and triggers fail-stop termination with coordinate details.

Syntax Analysis: Bison LALR(1) Parser (parser.y)

Grammar Verification & AST Generation

- **LALR(1) Shift-Reduce Engine:** Implements context-free grammar with unambiguous operator precedence (%left OR, %left AND, %left comp_ops).
- **Semantic Actions:** Instantiates AST nodes directly during grammar reductions (\$\$ = new SelectQueryNode(...)).
- **Detailed Diagnostics:** Enabled %define parse.error detailed to emit informative syntax errors (e.g., unexpected token, missing identifier).

Real Token Stream Example: SELECT name, cgpa FROM students WHERE cgpa > 8.0;

Verified Output from lexer.l

Emitted Tokens: [SELECT, KEYWORD] → [IDENTIFIER(name)] → [COMMA] → [IDENTIFIER(cgpa)] → [FROM, KEYWORD] → [IDENTIFIER(students)] → [WHERE, KEYWORD] → [IDENTIFIER(cgpa)] → [GREATER, >] → [FLOAT_LITERAL(8.0)] → [SEMICOLON]

Every token preserves its original source lexeme and line coordinates, allowing the visualizer to highlight query tokens dynamically during execution.

Frontend Compilation: AST Hierarchy & Semantic Analysis

Abstract Syntax Tree (ast.h, ast.cpp)

Object-Oriented Representation

- **Node Hierarchy:** Base ASTNode with derived SelectQueryNode and ExprNode subclasses (BinaryOpNode, ColumnRefNode, LiteralNode).
- **AST Decoupling:** Fully isolates semantic query meaning from parser-specific grammar artifacts and punctuation.
- **Dual Output Capabilities:** Provides recursive ASCII tree printing for terminal debugging and JSON serialization for web UI rendering.

Symbol Table & Semantic Analysis

Catalog Verification & Type Checking

- **Catalog Schema:** Maintains schema for relation 'students': id (INT), name (STRING), cgpa (FLOAT), age (INT).
- **6-Point Semantic Checklist:** (1) Table existence, (2) SELECT column validation, (3) WHERE column validation, (4) ORDER BY column validation, (5) Operator type compatibility, (6) Boolean predicate check.
- **Strict Type Enforcement:** Rejects comparisons between incompatible types (e.g. comparing FLOAT with STRING literal).

Semantic Verification Examples (Verified in Code)

Catalog Validation Scenarios

- **Valid Query Check:** 'SELECT name, cgpa FROM students WHERE cgpa > 8.0;' → PASS (table found, columns verified, FLOAT > FLOAT valid).
- **Invalid Attribute Error:** 'SELECT salary FROM students;' → REJECTED at Stage 06: Column 'salary' does not exist in table 'students'.
- **Invalid Type Error:** 'SELECT name FROM students WHERE cgpa > 'Alice';' → REJECTED at Stage 06: Type mismatch: cannot compare FLOAT with STRING.

Intermediate Representation: Relational Algebra Trees

Why Relational Algebra IR? (ir.h, ir.cpp)

Abstracting Execution from Surface Syntax

- **Separation of Concerns:** AST reflects concrete query syntax; Relational IR translates syntax into formal relational operators independent of execution mechanics.
- **Canonical Operator Tree:** Generated systematically from validated ASTs using standard algebraic mapping.
- **Optimization Foundation:** Algebraic rewriting rules operate directly on IR nodes, enabling equivalence transformations.
- **Implemented Relational IR Nodes:**
 - **ScanNode:** Reads source relation from CSV storage.
 - **FilterNode:** Evaluates selection condition (σ_p) per tuple.
 - **SortNode:** Orders tuples by key column (τ) in ASC/DESC order.
 - **ProjectNode:** Restricts attributes to requested columns (π_A).
 - **LimitNode:** Enforces maximum tuple output cardinality.

Canonical IR Plan Tree

Query: SELECT name, cgpa FROM students WHERE cgpa > 8.0 ORDER BY cgpa LIMIT 3;

▼ LIMIT (3)

Caps output tuples to at most 3 rows

▼ PROJECT [name, cgpa]

Restricts tuple attributes to selected projection columns

▼ SORT [cgpa ASC]

Sorts intermediate qualifying tuples in ascending order

▼ FILTER [cgpa > 8.0]

Evaluates float comparison predicate per incoming row

▼ SCAN [students]

Sequential scan loading base tuples from students.csv

Query Optimization: Rule-Based Algebraic Rewrites

Implemented Rule-Based Optimizer (optimizer.h, optimizer.cpp)

Algebraic Transformation Principles

- **Rule 1 - Filter Pushdown:** Pushes selection predicates (σ) down directly onto ScanNode to eliminate non-qualifying tuples before expensive operations. If already adjacent to scan, optimizer explicitly flags 'NOT REQUIRED'.
- **Rule 2 - Late Projection (Projection Reordering):** In canonical plans, PROJECT occurs before SORT. However, if ORDER BY references a column not in SELECT, early projection strips the sort key, causing query failure. The optimizer pushes PROJECT above SORT to preserve the sort key.
- **Rule 3 - Limit Early-Stopping:** When no ORDER BY clause is present, LIMIT is pushed directly into ScanNode, stopping file I/O immediately once N qualifying rows are emitted.

Canonical Plan (Before Rewrite)

Query: SELECT name, age FROM students WHERE age >= 20 ORDER BY cgpa LIMIT 3;

- LIMIT (3)
- SORT (cgpa ASC) <-- Sort needs 'cgpa'
- **PROJECT [name, age] <-- Problem:** Strips 'cgpa' here!
- FILTER (age >= 20)
- SCAN (students)
- **Status:** Incorrect plan if executed naively without optimization.

Optimized Plan (After Late Projection)

Rule Applied: Projection Reordering

- LIMIT (3)
- PROJECT [name, age] <-- Pushed ABOVE Sort!
- SORT (cgpa ASC) <-- Retains 'cgpa' key safely!
- FILTER (age >= 20)
- SCAN (students)
- **Benefit:** Guarantees relational correctness while keeping tuple width minimal.

Physical Query Execution Engine & CSV Storage

Execution Pipeline (executor.h, executor.cpp)

In-Memory Streaming CSV Processing

- **CSV File Streaming:** Reads data/students.csv row-by-row, validating headers against schema.
- **Row Deserialization:** Parses string fields into typed runtime variants (int, float, string).
- **Expression Evaluation:** Evaluates boolean AST expressions dynamically for each candidate tuple.
- **In-Memory Sorting:** Executes stable quicksort on tuples matching the ORDER BY column and direction.
- **Column Projection:** Prunes unwanted columns from qualifying tuples to match the SELECT clause.
- **Cardinality Restriction:** Emits up to LIMIT rows and halts execution cleanly.
- **High-Resolution Profiling:** Measures nanosecond execution times for each pipeline stage.

Verified Execution Trace & Result Output

Query: SELECT name, age FROM students WHERE age >= 20 ORDER BY cgpa LIMIT 3;

Step-by-Step Physical Execution Trace:

- SCAN students → 5 base rows loaded from CSV
- FILTER age >= 20 → 3 qualifying rows retained (Alice, Bob, David)
- **SORT by cgpa ASC → Ordered:** David (6.9), Bob (7.8), Alice (8.5)
- PROJECT [name, age] → Retained 2 columns per tuple
- LIMIT 3 → Emitted 3 final tuples

Result Dataset (Verified from minisql.exe):

- **Row 1:** name = 'David', age = 22 (cgpa 6.9)
- **Row 2:** name = 'Bob', age = 21 (cgpa 7.8)
- **Row 3:** name = 'Alice', age = 20 (cgpa 8.5)

Stage-Specific Error Handling & Fail-Stop Isolation

Fail-Stop Diagnostic Architecture

Strict Compiler Stage Error Isolation

- **Core Educational Principle:** Errors are halted at the exact compiler phase where they originate. Subsequent phases are never executed.
- **Standardized Error Categories:** Lexical, Syntax, Semantic, Type, and Execution errors are clearly classified with exact source line locations.

Error Category	Detecting Stage	Sample Input Query	Observed Diagnostic Message
Lexical Error	Stage 02 (Flex)	SELECT @name FROM students;	[LEXICAL ERROR] Unrecognized character '@' at line 1, column 8
Syntax Error	Stage 03 (Bison)	SELECT FROM students;	[SYNTAX ERROR] syntax error, unexpected 'FROM', expecting identifier
Semantic Error	Stage 06 (Semantic)	SELECT salary FROM students;	[SEMANTIC ERROR] Column 'salary' does not exist in table 'students'
Type Error	Stage 06 (Semantic)	SELECT name FROM students WHERE cgpa > 'Alice';	[TYPE ERROR] Incompatible types for operator '>': FLOAT and STRING
Execution Error	Stage 09 (Executor)	Corrupted CSV or missing file	[EXECUTION ERROR] CSV file not found or numeric parsing failed

Interactive Web Visualizer Workbench

Visualizer Architecture & Technical Stack

Local Development Workbench (Port 5173 / Port 3001)

- **Frontend Technology:** React 18, TypeScript, Tailwind CSS, Lucide icons, Vite 5 dev server.
- **Backend API Bridge:** Node.js Express server with `child_process` execution of `minisql.exe --json`.
- **100% Real C++ Execution:** Zero client-side simulated data. All stages reflect authentic C++ compiler JSON output.
- **Responsive Local Host:** Runs locally at `http://localhost:5173` via reverse proxy to backend at `http://localhost:3001`.
- **Strict Component Separation:** Dedicated React components for Tokens Table, AST Viewer, IR Viewer, Optimization Viewer, and Trace Logs.

Interactive Classroom & Viva Features

Educational Inspection Capabilities

- **10-Stage Animated Pipeline Stepper:** Provides Play, Pause, Step-Through, Skip, and Reset buttons to observe query compilation progressively.
- **Interactive Token Table:** Displays token type badges, source lexemes, and line numbers.
- **Collapsible AST Tree:** Tree visualizer expanding SELECT clauses, binary operator expressions, and column references.
- **Side-by-Side IR Comparison:** Direct visual diff between Canonical IR and Optimized Plan with rule explanation badges.
- **Live Execution Trace & Results Table:** Dynamic tabular rendering of CSV output with row counts and stage timings.
- **Built-in Viva & Theory Guide Modal:** On-demand reference modal providing compiler theory questions and answers for oral examination.

Automated Testing & Experimental Validation

Automated Test Suite Summary (run_tests.ps1)

26 Tests Executed Against minisql.exe — 100% Success Rate

- **Systematic Test Harness:** Automated PowerShell test script executing all test queries against minisql.exe.
- **Coverage Scope:** Valid projections, float/int comparisons, boolean logic, ordering, limits, boundary cases, error isolation, and JSON bridge output.

Category	Test IDs	Scenario Description	Expected Behavior	Status
Valid Projections	T01, T02	Single & multi-column selection	Extracts requested columns cleanly	PASS
Predicate Filters	T03, T04	Float (cgpa > 8.0) & Int (age >= 20)	Filters tuples matching condition	PASS
Logical Operators	T05, T06	Boolean AND / OR conjunctions	Evaluates compound predicates	PASS
Ordering & Limits	T07, T08, T09	ORDER BY, LIMIT, and combined	Sorts tuples & caps row count	PASS
Late Projection	T10, T11	ORDER BY on unselected column	Optimizer reorders projection above sort	PASS
Boundary Cases	T12, T13, T14	LIMIT 0, LIMIT > rows, LIMIT w/o sort	Safe early termination	PASS
Error Isolation	T17–T22	Lexical, Syntax, Semantic, Type errors	Halts at exact originating stage	PASS
API Bridge Output	T26	minisql.exe --json bridge execution	Valid machine-readable JSON	PASS

Module-Wise Implementation Progress Matrix

Module / Subsystem	Technology / File	Project Phase	Review 2 Status	Verification Method
Lexical Scanner	Flex (lexer.l)	Phase 1	100% Complete	Verified with test queries & error traps
Syntax Parser	Bison (parser.y)	Phase 1	100% Complete	Verified via LALR(1) reduction tests
AST Construction	C++17 (ast.cpp)	Phase 1	100% Complete	ASCII tree print & JSON export verified
Symbol Catalog	C++17 (symbol_table.cpp)	Phase 1	100% Complete	Catalog schema lookup verified
Semantic Analyzer	C++17 (semantic_analyzer.cpp)	Phase 1	100% Complete	6-point semantic checklist passing
Relational IR	C++17 (ir.cpp)	Phase 2	100% Complete	Canonical operator trees generated
Query Optimizer	C++17 (optimizer.cpp)	Phase 2	100% Complete	Filter pushdown & late projection verified
Physical CSV Executor	C++17 (executor.cpp)	Phase 2	100% Complete	In-memory tuple evaluation verified
Fail-Stop Error Traps	C++ Pipeline Traps	Phase 2	100% Complete	Verified via T17–T25 test cases
CLI JSON Bridge	minisql.exe --json	Phase 2 / 3	100% Complete	Pure structured JSON emitted to stdout
Web Visualizer UI	React 18 / TypeScript	Phase 3	~80% Complete	Functional locally on port 5173
Multi-Table Joins	Relational Join IR	Phase 3	Pending Scope	Reserved for Phase 3 extension

Review 2 Assessment Rubric & Evidence Mapping

Rubric Criteria	Marks	Project Implementation Evidence	Deliverable Status
Implementation Progress	7 Marks	Complete working C++ pipeline (Lexer → Parser → AST → Semantic → IR → Optimizer → Executor) plus functional React/Node web visualizer.	Demonstrated via live binary execution
Technical Correctness	5 Marks	Accurate LALR(1) parsing, type checking, operator tree construction, and CSV query evaluation verified across 26 automated tests.	Verified: 26/26 tests passing (100%)
Compiler Concept Application	4 Marks	Application of DFAs (Flex), context-free grammars (Bison), AST, symbol tables, intermediate relational code, and algebraic rewrites.	Demonstrated across 10 compiler stages
Code Quality & Modularity	3 Marks	Modular C++17 header/source separation, clean OOP inheritance, proper memory cleanup, and typed React components.	Verified in source repository
Testing & Test Cases	3 Marks	Systematic test suite (run_tests.ps1) testing projections, filters, logic, ordering, limits, and stage-specific error traps.	Demonstrated with test suite execution
Individual Understanding	3 Marks	Thorough comprehension of all compiler phases, AST traversal, relational algebra mapping, and oral viva readiness.	Demonstrated during oral viva review

Related Academic Work in Compiler Education (2020–2025)

Authors & Year	Venue / Journal	Key Findings & Contribution	Comparison with SQLens
Stamenković et al. (2020)	CAEE (Wiley)	Evaluated 15 compiler simulation tools; identified lack of unified end-to-end visualization.	SQLens provides a unified 10-stage pipeline with web visualization.
del Vado Vírveda (2021)	ACM ITiCSE	Proposed learning compiler design from implementation to theory using educational grammars.	SQLens adopts this implementation-first approach for relational SQL.
del Vado Vírveda (2022)	ACM SIGCSE	Developed ITT interactive tutoring tool for visualizing compiler theory from implementation.	SQLens extends tutoring concepts to database query compilers and relational IR.
Hiranishi et al. (2022)	IEEJ Trans. EIS	Developed web-based visual compiler for computer education to visualize execution flow.	SQLens pairs web visualization with a native compiled C++ core engine.
Stamenković & Jovanović (2024)	IEEE TLT	Demonstrated that web educational compiler systems significantly improve student engagement.	SQLens confirms the efficacy of browser-accessible interactive compiler tools.
Koitz-Hristov et al. (2025)	ACM SIGCSE TS	Introduced VisOpt for visualizing compiler optimization passes in computer science education.	SQLens focuses on relational query optimizations (filter pushdown & late projection).

What Distinguishes SQLens? An Honest Assessment

What SQLens Does NOT Claim

Realistic Academic Framing

- **Not a Novel Pedagogical Invention:** Visual compiler simulators and educational tools have been studied since the early 2000s; visualization itself is not a new research discovery.
- **Not a Production Database Engine:** SQLens does not attempt to replace commercial or open-source engines like PostgreSQL, DuckDB, or SQLite.
- **Not a High-Performance Query Processor:** MiniSQL is designed for maximum clarity, explainability, and pedagogical inspection, not multi-threaded micro-second throughput.

Genuine Engineering Contributions of SQLens

Concrete Project Innovations

- **Cross-Disciplinary Synthesis:** Unifies standard Compiler Design frontend techniques (Flex/Bison) with DBMS backend concepts (Relational IR, Query Optimization, CSV Execution) in one tool.
- **Zero-Mock Native Architecture:** The web visualizer is 100% driven by real C++ compiled binary outputs emitted via structured JSON—no simulated mockups.
- **Multi-Stage Diagnostic Isolation:** Enforces strict fail-stop boundaries so students immediately recognize whether an error is Lexical, Syntactic, Semantic, or Runtime.
- **Relational Plan Equivalence Inspection:** Visually demonstrates canonical vs. optimized algebraic trees (e.g. Late Projection) for classroom learning.

Future Work & Computing Education Research Potential

Engineering Roadmap (Phase 3 Extensions)

Planned Technical Enhancements

- **Multi-Table Relational Joins:** Implementing Scan-Join trees, Nested Loop Joins, and Hash Join algorithms for multi-relation queries.
- **Aggregate Expressions:** Adding COUNT(), SUM(), AVG(), MIN(), MAX(), and GROUP BY / HAVING clauses to the grammar and execution engine.
- **Additional Algebraic Passes:** Implementing constant folding in WHERE predicates and dead attribute elimination in projection.
- **Execution Stepping Animation:** Visualizing row-by-row filtering in the UI, showing qualifying rows highlighted and pruned rows grayed out.
- **Automated Benchmark Suite:** Measuring execution times across scaling CSV sizes (100 to 10,000 tuples).

Research Potential (ACM SIGCSE Venue)

Educational Impact Evaluation Strategy

- **Target Conference Venue:** ACM Technical Symposium on Computer Science Education (SIGCSE) — premier computing education venue.
- **Proposed Research Question:** 'Does an integrated, stage-by-stage visualization of a real SQL compiler improve undergraduate students' conceptual understanding and debugging accuracy compared to traditional instruction?'
- **Empirical Study Design:** Pre-test and post-test controlled classroom study evaluating student comprehension of ASTs, IR trees, and optimization passes.
- **Realistic Publication Prerequisite:** A working software tool alone is not a research paper; rigorous empirical evaluation with human student participants is required before submission.

Conclusion & Live Review Demonstration Plan

Review 2 Summary of Achievements

Current Milestone Overview

- **Fully Operational C++ Core:** Complete MiniSQL compiler engine implemented with Flex, Bison, and standard C++17.
- **10-Stage Pipeline:** Clean realization of lexical analysis, LALR(1) parsing, AST, symbol catalog, semantic checking, relational IR, algebraic optimization, and CSV execution.
- **Strict Fail-Stop Error System:** Verified error isolation across lexical, syntax, semantic, type, and runtime categories.
- **26/26 Automated Tests Passing:** 100% test pass rate across all functional and boundary test suites.
- **Integrated Web Workbench:** Fully functional local React/Node visualizer connected directly to minisql.exe.

Live Review Demonstration Sequence

Planned Viva Demonstration Steps

- **Step 1 - The Grand Tour Query:** Execute 'SELECT name, age FROM students WHERE age >= 20 ORDER BY cgpa LIMIT 3;' to showcase all 10 stages from tokens to result.
- **Step 2 - Relational Optimization Inspection:** Inspect side-by-side Canonical vs. Optimized IR plan and verify the Late Projection rule rewrite.
- **Step 3 - Semantic Fail-Stop Demonstration:** Execute 'SELECT salary FROM students;' to prove pipeline halt at Stage 06 with non-existent column diagnostic.
- **Step 4 - Lexical Error Demonstration:** Execute 'SELECT @name FROM students;' to prove front-end lexical character rejection at Stage 02.
- **Step 5 - Automated Test Suite:** Execute .\run_tests.ps1 in PowerShell to demonstrate 26/26 passing automated tests.

Academic References & Project Bibliography

Peer-Reviewed Academic Literature

Primary Scholarly Works

- [1] Stamenković, D., Jovanović, Z., & Chakraborty, P. (2020). Evaluation of simulation systems suitable for teaching compiler construction courses. *Computer Applications in Engineering Education*, 28(5), 1160–1177. DOI: 10.1002/cae.22231
- [2] del Vado Vírveda, R. (2021). *Learning Compiler Design: From the Implementation to Theory*. In Proc. 26th ACM Conf. Innovation and Technology in Computer Science Education (ITICSE 2021), pp. 282–288. DOI: 10.1145/3456565.3460041
- [3] del Vado Vírveda, R. (2022). *ITT: An Interactive Tutoring Tool to Improve the Learning and Visualization of Compiler Design Theory From Implementation*. In Proc. 53rd ACM Technical Symposium on Computer Science Education (SIGCSE 2022), pp. 696–702. DOI: 10.1145/3478432.3499140
- [4] Hiranishi, Y., et al. (2022). *Application of Web-based Visual Compiler to Computer Education*. *IEEJ Trans. EIS*, 142(4), 389–396. DOI: 10.1541/ieejtrans.142.389

Recent Work, Texts & University Guidelines

Supplementary Citations

- [5] Stamenković, D., & Jovanović, Z. (2024). *A Web-Based Educational System for Teaching Compilers*. *IEEE Transactions on Learning Technologies*, 17, 819–830. DOI: 10.1109/TLT.2023.3297626
- [6] Koitz-Hristov, R., Mandl, M., & Wotawa, F. (2025). *VisOpt – Visualization of Compiler Optimizations for Computer Science Education*. In Proc. 56th ACM Technical Symposium on Computer Science Education (SIGCSE TS 2025), Pittsburgh, PA, USA.
- [7] School of Computer Science and Engineering, Vellore Institute of Technology. (2026). *Compiler Design Laboratory Project Instruction Manual (BCSE307P)*, VIT Vellore, India.
- [8] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.